

Tr_12 – Diagrama de Clases UML

Documento de Referencia: **Diagrama de Clases UML**

Proyecto: Aplicación “**No Te Cortes**”

Sector: **Servicios de Estética e Imagen Personal (CNAE 9602)**

1. Justificación Técnica

El diagrama representa la estructura estática de la aplicación utilizando la notación UML 2.5. Se ha diseñado una arquitectura basada en POJOs (*Plain Old Java Objects*) para facilitar la persistencia en bases de datos NoSQL como Firebase Firestore.

Se han aplicado los siguientes principios de Programación Orientada a Objetos (POO):

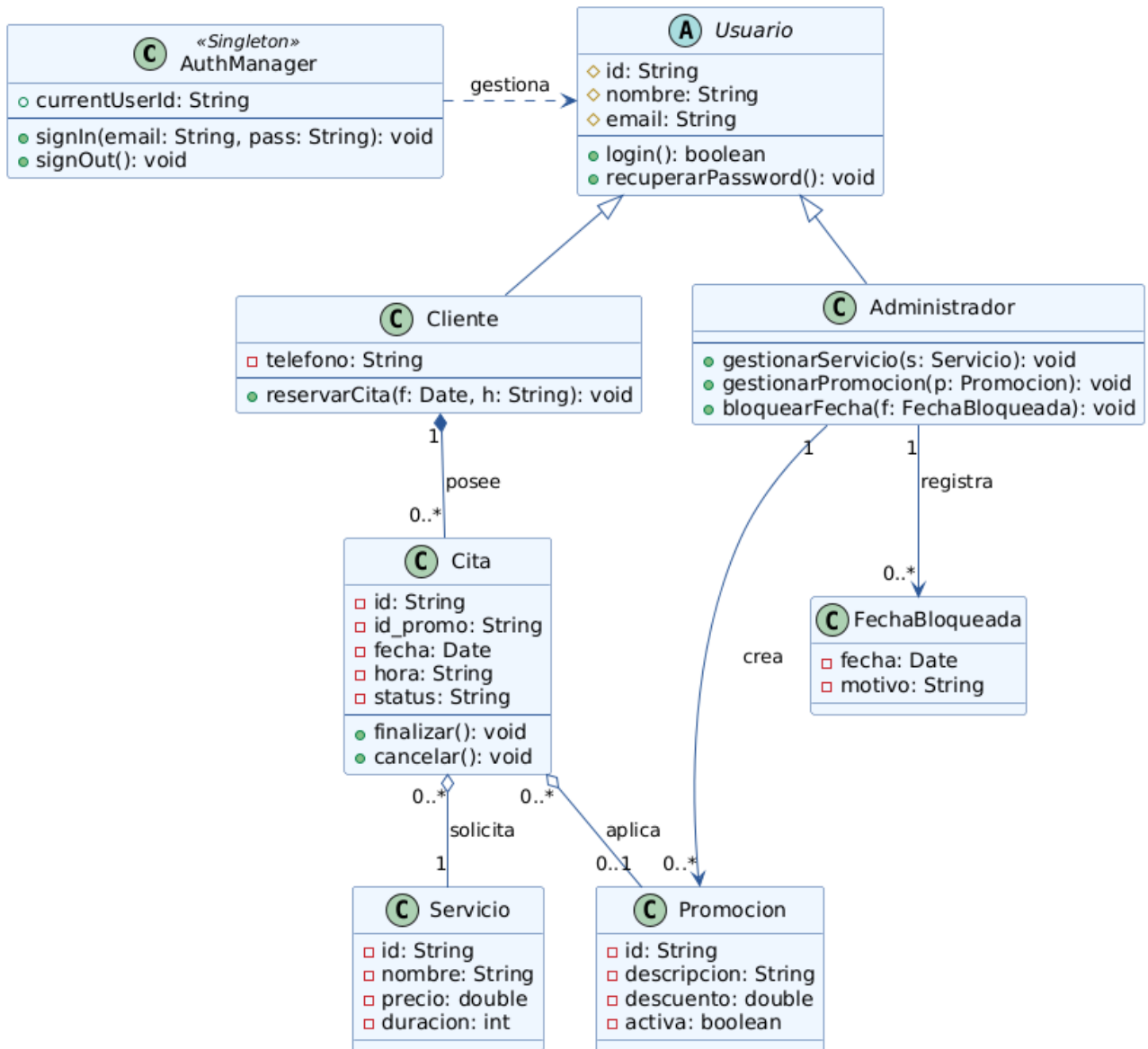
- **Encapsulamiento:** Uso de visibilidad privada (-) para proteger los datos y pública (+) para los métodos de interfaz.
- **Abstracción:** La clase Usuario se define como abstracta, ya que no se instancian usuarios genéricos, sino perfiles específicos.
- **Especialización:** Uso de herencia para que Cliente, Empleado y Administrador compartan la lógica base de sesión.

2. Definición de Clases y Relaciones

Para este diseño, aplicamos una arquitectura basada en Modelos (POJOs) y Controladores.

- **Herencia:** Centralizamos en Usuario la lógica de sesión y datos básicos. Cliente, Empleado y Administrador especializan las acciones.
- **Agregación (Cita - Servicio):** Una Cita necesita un Servicio. Es agregación porque el servicio existe independientemente de si hay citas o no.
- **Agregación (Cita - Promoción):** Una Cita puede incluir de forma opcional una Promoción activa para el cálculo final del precio. Al igual que los servicios, la promoción existe de forma autónoma en el catálogo.
- **Composición (Usuario - Cita):** En el contexto de nuestra App, una Cita no tiene sentido sin un Usuario (Cliente) que la posea.

3. Diagrama de Clases



4. Descripción de Componentes

A. Atributos y Métodos

- **Clase Cita:** Incluye atributos técnicos como id, fecha, hora, status y la referencia opcional a la promoción activa..
- **Clase Administrador:** Posee métodos de gestión específicos con parámetros definidos: gestionarServicio(s: Servicio) y gestionarPromocion(p: Promocion).
- **Clase AuthManager:** Se implementa como Singleton para garantizar una única instancia de gestión de sesión en toda la App.

Nota de diseño: La entidad PASSWORD_RESET definida previamente en el modelo conceptual se ha integrado en este diagrama mediante el método recuperarPassword() dentro de la clase Usuario. Esta decisión busca simplificar la lógica de negocio y aprovechar la integración nativa de Firebase Auth para la gestión de credenciales.

Nota de implementación financiera: Aunque en el diagrama de clases estático se definen los campos de corte económico (precio y descuento) como 'double' por compatibilidad estándar de tipado en interfaces UML, en el entorno de desarrollo real en Java se implementarán mediante la clase de alta precisión 'BigDecimal'. De este modo se garantiza la coherencia exacta con los tipos 'DECIMAL(10,2)' proyectados en el esquema relacional y se previenen errores de redondeo en los flujos de caja.

B. Relaciones y Cardinalidad

- **Cliente - Cita** : Un cliente puede no tener citas o tener muchas. Cada cita pertenece obligatoriamente a 1 cliente.
- **Cita - Servicio** : Una cita solicita exactamente 1 servicio. Un servicio específico puede estar presente en muchas citas diferentes.
- **Cita - Promocion** : Una cita puede no aplicar ninguna promoción o aplicar un máximo de una (1) para bonificar su coste. Una promoción puede estar presente en múltiples citas concurrentes.
- **Administrador - Promoción / FechaBloqueada** : Un administrador registra y gestiona múltiples promociones y bloqueos de calendario. La clase FechaBloqueada incluye los atributos 'fecha' y 'motivo', mapeando de forma consistente con la lógica del negocio. Las restricciones de integridad (como evitar que se borre un servicio con citas activas) se controlarán mediante funciones de validación en los controladores Java, supliendo la ausencia de claves foráneas nativas en Cloud Firestore.